

Rapport TP9 : les Structures de Données

Introduction

Une structure de donnée est une manière de stocker et organiser les données dans un ordinateur de manière à ce qu'il soit utilisé de façon optimale.

On parlera de structure de donnée comme un modèle logique (mathématique) ou de type (structure) de donnée abstrait.

Les structures de données abstraites définissent les données utilisées et les opérations effectuées sur ces données.

Pour chaque type de structure de donnée abstrait (abstract data type) sont associés plusieurs implémentations définissant comment les structures de données sont construites dans un ordinateur, ces implémentations sont dites structures de données concrètes.

Par exemple, les LIST sont des structures de données abstraites caractérisées comme suit :

- Gère tous les types de données
- Sur ces données peuvent s'appliquer les opérations suivantes : opérations de stockage, de lecture, de modifications des données
- Quelques implémentations des listes : Array (Tableau), Linked List, double linked list etc.

Lors de l'étude de ces structures de données, nous allons nous attarder sur les structures de données concrètes, en ressortant pour chacun :

- une vue logique de la structure de donnée
- les opérations possibles sur la structure de donnée
- le coût des opérations : complexité temporelle (un ordre de grandeur du temps mis pour réaliser l'opération) et la complexité spatiale (un ordre de grandeur de la mémoire occupée pendant ces opérations)

1. Tableau (Array)

Définition

Collection d'éléments de même type stockés dans des emplacements mémoire contigus, accessibles par un index.

Cas d'utilisation

- **Accès de manière aléatoire fréquent**(les Tableau sont très recommander si ces opérations sont très fréquente sur vos structure de données et qu'en plus la taille de votre structure est fixe et connue à l'avance.)
- **Taille fixe connue à l'avance**
- **Opérations mathématiques vectorielles**
- **Insertions/suppressions fréquentes** (si ces opérations sont fréquentes sur vos structures de données , il n'est pas conseiller d'utiliser les Tableau car le coût des opérations est moins favorable par rapport à certains autres structure de données existantes)

Complexité Temporelle des opérations :

- **Accès $O(1)$** : ici on a un temps constant proportionnel a la durée d'exécution d'une instructions,

pour une complexité temporelle **$O(1)$** on peut par exemple avoir un temps d'exécution **$T = C \times 1U = C$** ou C est une constante et 1U (pris comme base d'unité temporelle) est le temps mis pour exécuter par exemple une opération logique(addition, soustraction etc.) par la machine,
pour un temps $T = C \times 1U = C$ on a donc une complexité temporel $O(1)$

- **Insertion $O(n)$** , ici on a un temps d'exécution qui peut avoir la forme suivante : **$T(n) = An + B$** ou A et B sont des constante

- **Recherche $O(n)$** , ici on a de même un temps d'exécution qui peut avoir la forme suivante : **$T(n) = An + B$** ou A et B sont des constante

Complexité spatiale des opérations(espace mémoire occupé pendant l'exécution de l'opération) :

pour tous ces opérations, dépendamment de l'algorithme on peut avoir une complexité spatiale **$=O(n)$** .

L'espace mémoire occupé à donc la forme suivante : **$S(n) = (An + B) \times 1U = An + B$** ou 1U (pris comme base d'unité d'espace mémoire) est par exemple la taille de l'espace mémoire nécessaire pour stocker un élément du tableau

Algorithme associer aux différentes opérations

* Accès_Element

```
Fonction Accès_Element(Tableau T, Indice i) : Element
    // Vérifier si l'indice est valide
    Si i >= 0 ET i < Taille(T) Alors
        Retourner T[i]
    Sinon
        Afficher "Erreur : Indice hors limites"
        Retourner Null // Ou une valeur d'erreur appropriée
    Fin Si
Fin Fonction
```

on voit ici que quelque soit la taille du tableau on a un nombre constant d'opération effectuée d'où une complexité temporelle **$O(1)$** . Pour cet algorithme on a maximum 3 instructions effectuée (comparaison , retour de valeur etc,)

* Recherche_Sequentielle

```
Fonction Recherche_Sequentielle(Tableau T, Valeur V) : Indice
    Pour i allant de 0 à Taille(T) - 1
        Si T[i] == V Alors
            Retourner i // L'élément est trouvé, retourner son indice
        Fin Si
    Fin Pour
    Retourner -1 // L'élément n'a pas été trouvé
Fin Fonction
```

ici on constate qu'au pire des cas c'est à dire si l'élément se trouve à la dernière position , on aura un temps = n fois le temps associées aux instructions d'une boucle d'où la complexité temporelle **$O(n)$** . mais il existe d'autre algorithme de recherche comme le **binary search** qui permette d'avoir une complexité temporelle bien moindre de l'ordre de **$O(\log n)$**

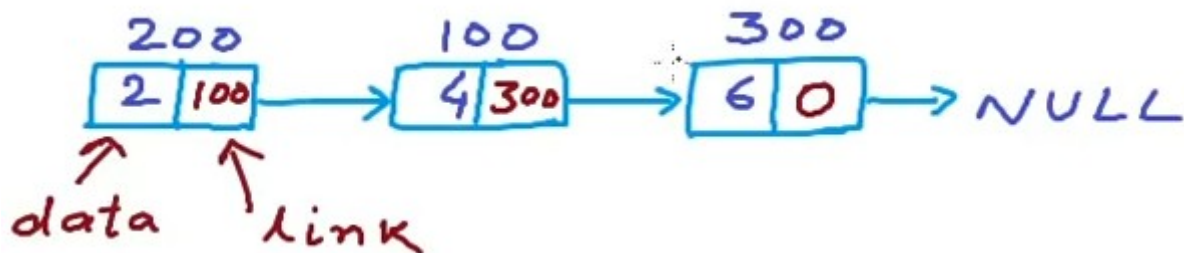
* Insertion_Position

```
Procedure Insertion_Position(Tableau T, Indice Pos, Valeur V)
  Si Taille(T) < Taille_Max(T) Alors // Vérifier s'il y a de la place
    // Décaler les éléments vers la droite à partir de la fin jusqu'à
    Pour i allant de Taille(T) - 1 à Pos descendant
      T[i + 1] = T[i]
    Fin Pour
    // Insérer la nouvelle valeur
    T[Pos] = V
    // Mettre à jour la taille (si nécessaire, selon l'implémentation)
    Incrementer Taille(T)
  Sinon
    Afficher "Erreur : Tableau plein, impossible d'insérer"
  Fin Si
Fin Procedure
```

2. Liste Chaînée (Linked List)

Définition

Collection d'éléments (nœuds) où chaque nœud contient des données et un pointeur vers le nœud suivant. Le pointeur stocke l'adresse du nœud suivant, ce qui établit le lien entre les deux nœuds



Cas d'utilisation

- Insertions/suppressions fréquentes
- Taille dynamique
- Implémentation de piles/files
- Accès aléatoire rare

Complexité: Insertion $O(1)$, Accès $O(n)$, Recherche $O(n)$

Algorithme associer aux différentes opérations

```
STRUCTURE Nœud
  données: type
  suivant: pointeur Nœud

DÉBUT ListeChâinée
  tête: pointeur Nœud

  MÉTHODE insertionDébut(valeur)
    nouveau = nouveau Nœud(valeur)
    nouveau.suivant = tête
    tête = nouveau

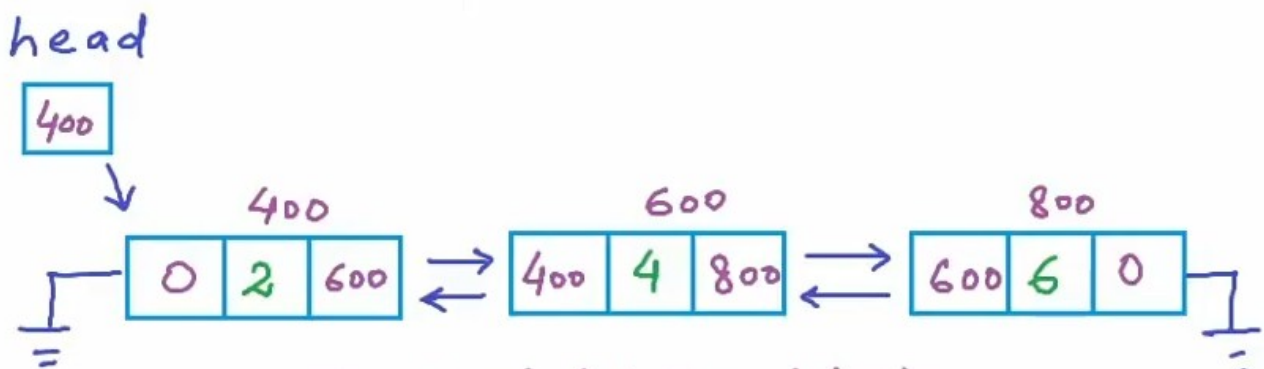
  MÉTHODE suppressionDébut()
    SI tête ≠ NULL
      temp = tête
      tête = tête.suivant
      libérer temp

FIN
```

3. Liste Doublement Chaînée (Doubly Linked List)

Définition

Liste chaînée où chaque nœud contient des pointeurs vers le nœud précédent et suivant.



Cas d'utilisation

- Navigation avant/arrière nécessaire

- Implémentation de caches (LRU)
- Systèmes d'historique
- Mémoire supplémentaire pour les pointeurs

Complexité: Insertion $O(1)$, Suppression $O(1)$, Accès $O(n)$

Algorithme associer aux différentes opérations

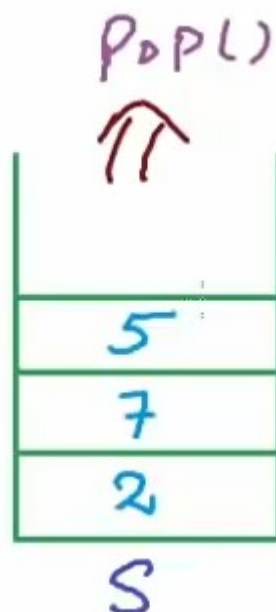
```
STRUCTURE NœudDouble
    données: type
    précédent: pointeur NœudDouble
    suivant: pointeur NœudDouble

MÉTHODE insertionMilieu(nœudExistant, valeur)
    nouveau = nouveau NœudDouble(valeur)
    nouveau.suivant = nœudExistant.suivant
    nouveau.précédent = nœudExistant
    nœudExistant.suivant.précédent = nouveau
    nœudExistant.suivant = nouveau
```

4. ✂ Pile (Stack)

Définition

Structure de donnée abstraite souvent appelé structure LIFO (Last In, First Out) où les éléments sont ajoutés et retirés du sommet. son implementations concrète peut se faire en utilisant les Arrays ou les Linked lists



Opérations : Insertions(push(x)), supprimer(pop()), obtenir l'element au top de la stack (Top()), isEmpty() verifier si la stack est vide

Complexité: Toutes les opérations $O(1)$

Cas d'utilisation

- Appels de fonctions (call stack)
- Annulation/refaire (undo/redo)
- Évaluation d'expressions
- Parcours DFS
- Balanced parentheses : utiliser par le compilateur pour verifier les expression mathematique

Algorithme associer aux différentes opérations

```
DÉBUT Pile
  éléments: liste
  sommet: entier = -1

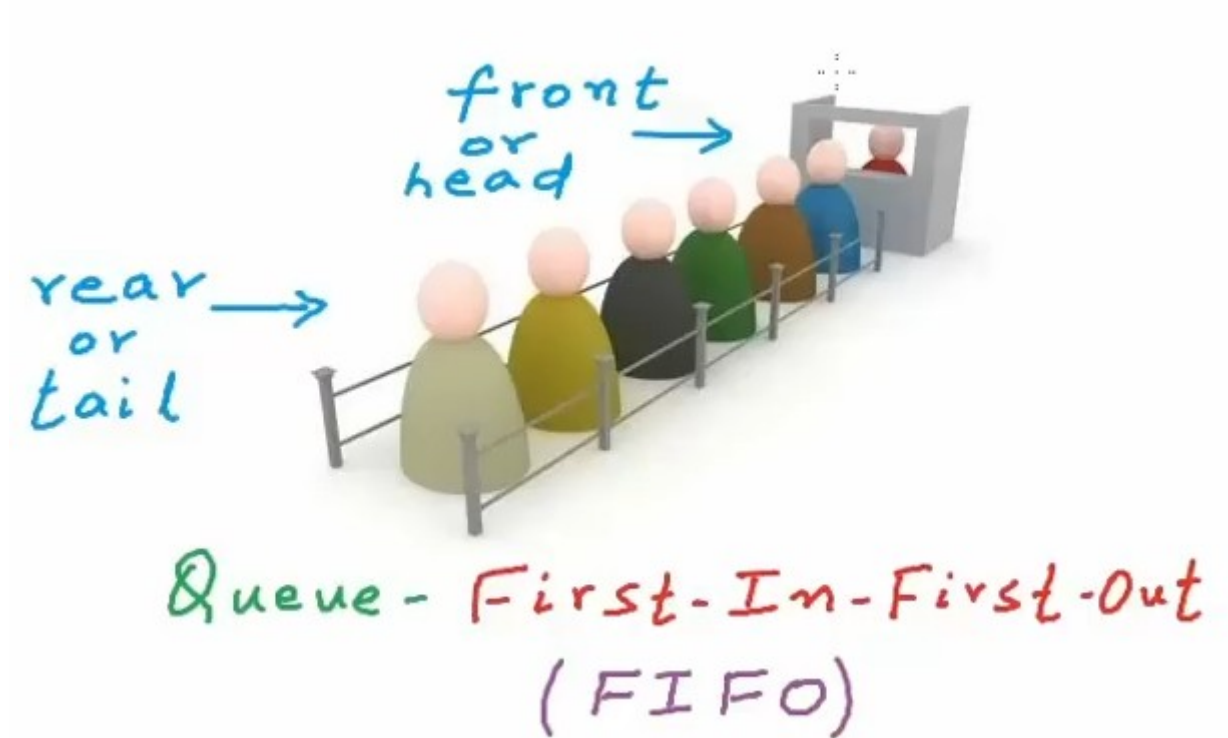
  MÉTHODE push(valeur)
    sommet = sommet + 1
    éléments[sommet] = valeur

  MÉTHODE pop()
    SI isEmpty() ALORS erreur
    valeur = éléments[sommet]
    sommet = sommet - 1
    RETOURNER valeur
FIN
```

5. File (Queue)

Définition

Structure de données abstraite souvent appelée structure FIFO (First In, First Out) où les éléments sont ajoutés à la fin et retirés du début.



Opérations : Insertions(Enqueue(x) or push(x)), supprimer(dequeue() or pop()), obtenir l'élément au top de la stack (front() or peek()), isEmpty() vérifier si la stack est vide

Complexité: Toutes les opérations $O(1)$

Cas d'utilisation

- Gestion de processus
- Traitement de requêtes
- Buffers d'E/S
- Recherche BFS

Algorithme associer aux différentes opérations

```
DÉBUT File
    éléments: liste
    tête: entier = 0
    queue: entier = -1

    MÉTHODE Enqueue(valeur)
        queue = queue + 1
        éléments[queue] = valeur

    MÉTHODE Dequeue()
        SI estVide() ALORS erreur
        valeur = éléments[tête]
        tête = tête + 1
        RETOURNER valeur

FIN
```

6. File Prioritaire (Priority Queue)

Définition

File où chaque élément a une priorité, les éléments de plus haute priorité sont servis en premier.

Cas d'utilisation

- Ordonnancement CPU
- Systèmes de tri
- Algorithmes de Dijkstra, Prim
- Traitement d'événements

Complexité: Insertion $O(\log n)$, Extraction $O(\log n)$

Algorithme associer aux différentes opérations

```
DÉBUT FilePrioritaire
    // Implémentée avec un tas (heap)
    tas: liste

    MÉTHODE insérer(élément, priorité)
        ajouter (élément, priorité) à tas
        réorganiser le tas vers le haut

    MÉTHODE extraireMax()
        max = tas[0]
        tas[0] = dernier élément
        réorganiser le tas vers le bas
        RETOURNER max

FIN
```

7. Tas (Heap)

Définition

Arbre binaire complet respectant la propriété de tas (tas max/min).

Cas d'utilisation

- Implémentation files prioritaires
- Algorithme de tri heapsort
- Sélection du k-ème élément
- Pas pour la recherche d'éléments

Complexité: Insertion $O(\log n)$, Extraction $O(\log n)$

Algorithme associer aux différentes opérations

```
DÉBUT TasMax
  données: liste

  MÉTHODE insertion(valeur)
    ajouter valeur à la fin
    index = taille - 1
    TANT QUE index > 0 ET parent(index) < valeur
      échanger avec parent
      index = parent(index)

  MÉTHODE extraireMax()
    max = données[0]
    données[0] = données[dernier]
    supprimer dernier
    réorganiserVersBas(0)
    RETOURNER max

FIN
```

8. Table de Hachage (Hash Table/Map/Dictionary)

Définition

Structure qui associe des clés à des valeurs via une fonction de hachage.

Cas d'utilisation

- Recherche rapide par clé
- Élimination de doublons
- Cache

- Pas pour les données ordonnées

Complexité: Insertion $O(1)$, Recherche $O(1)$ en moyenne

Algorithme associer aux différentes opérations

```
DÉFINIR fonctionHachage(clé) → index

DÉBUT TableHachage
    buckets: liste de listes
    capacité: entier

    MÉTHODE insérer(clé, valeur)
        index = fonctionHachage(clé) MOD capacité
        bucket = buckets[index]
        POUR chaque paire dans bucket
            SI paire.clé = clé ALORS
                paire.valeur = valeur
            RETOURNER
        ajouter nouvelle paire (clé, valeur) à bucket

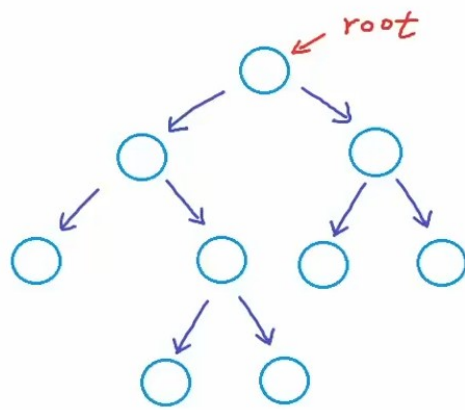
    MÉTHODE obtenir(clé)
        index = fonctionHachage(clé) MOD capacité
        bucket = buckets[index]
        POUR chaque paire dans bucket
            SI paire.clé = clé ALORS
                RETOURNER paire.valeur
        RETOURNER NULL

FIN
```

9. Arbre (Tree)

Définition

Structure hiérarchique avec un nœud racine et des sous-arbres.



Binary tree

↳ each node can have at most 2 children

Cas d'utilisation

- Systèmes de fichiers
- Bases de données
- DOM XML/HTML
- Structures organisationnelles

Complexité: Dépend de l'implémentation Arrays ou Linked list

10. Arbre Binaire de Recherche (Binary Search Tree - BST)

Définition

BST : c'est un Arbre Binaire ou pour chaque nœud pour lequel on a associé une valeur, tous les éléments du sous-arbre gauche ont des valeurs inférieurs a la valeur du nœuds de reference et ceux du droit ont des valeurs supérieurs.

Arbre Binaire : c'est un arbre ou chaque nœud a au plus 2 enfants

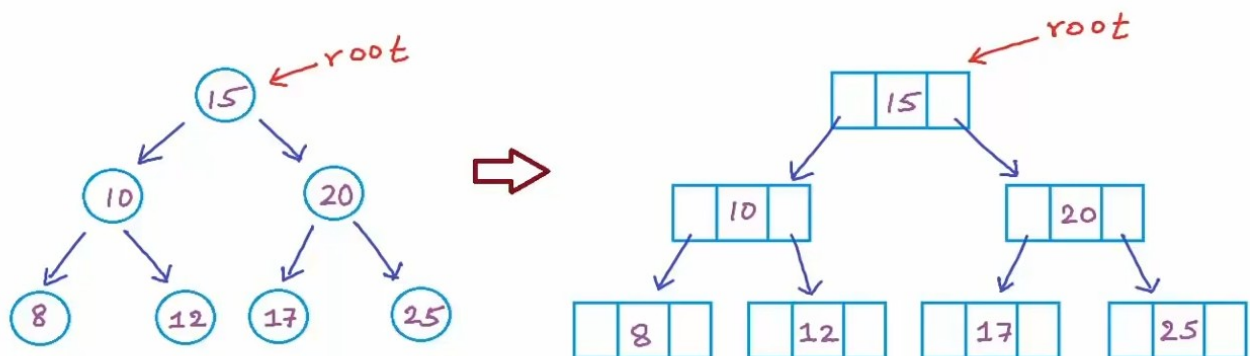


Image d'un BST et sont implementation inspirer des double linked list

Cas d'utilisation

- Recherche rapide dans données ordonnées
- Maintenance d'ensemble dynamique trié
- Peut devenir déséquilibré

Complexité: Recherche $O(\log n)$ en moyenne, $O(n)$ pire cas

Pseudo-algorithme

```
STRUCTURE NœudBST
    données: type
    gauche: pointeur NœudBST
    droit: pointeur NœudBST

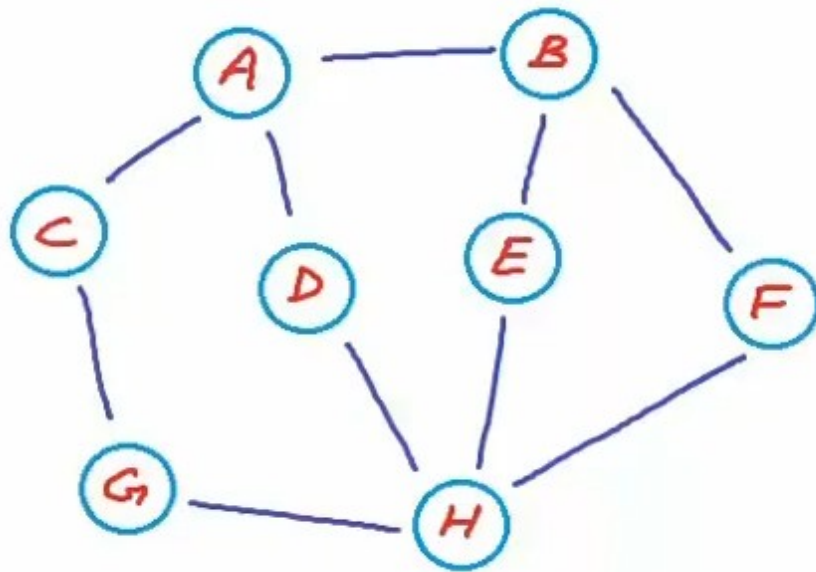
MÉTHODE recherche(racine, clé)
    SI racine = NULL OU racine.données = clé
        RETOURNER racine
    SI clé < racine.données
        RETOURNER recherche(racine.gauche, clé)
    SINON
        RETOURNER recherche(racine.droit, clé)

MÉTHODE insertion(racine, valeur)
    SI racine = NULL
        RETOURNER nouveau NœudBST(valeur)
    SI valeur < racine.données
        racine.gauche = insertion(racine.gauche, valeur)
    SINON
        racine.droit = insertion(racine.droit, valeur)
    RETOURNER racine
```

11. Graphe (Graph)

Définition

Ensemble de nœuds connectés par des arêtes (orientées ou non).



Cas d'utilisation

- Réseaux sociaux
- Systèmes de navigation
- Réseaux informatiques
- Dépendances entre tâches

Complexité: BFS $O(V+E)$, DFS $O(V+E)$

```
// Matrice d'adjacence
matrice: tableau 2D[taille][taille]

// Liste d'adjacence
STRUCTURE Graphe
  nœuds: liste
  listeAdjacence: liste de listes

MÉTHODE BFS(départ)
  file = nouvelle File()
  marqués = ensemble vide
  file.enfiler(départ)
  marqués.ajouter(départ)

  TANT QUE file non vide
    courant = file.défiler()
    TRAITER courant
    POUR chaque voisin de courant
      SI voisin non marqué
        file.enfiler(voisin)
        marqués.ajouter(voisin)
```

12. Ensemble (Set)

Définition

Collection d'éléments distincts sans ordre particulier.

Cas d'utilisation

- Vérification d'appartenance
- Opérations ensemblistes (union, intersection)
- Élimination de doublons
- Pas d'accès par index

Complexité: Ajout $O(1)$, Recherche $O(1)$ en moyenne

Pseudo-algorithme

```
// Implémentation avec table de hachage
DÉBUT Ensemble
    table: TableHachage

    MÉTHODE ajouter(élément)
        table.insérer(élément, vrai)

    MÉTHODE contient(élément)
        RETOURNER table.obtenir(élément) ≠ NULL

    MÉTHODE union(otherSet)
        résultat = nouvel Ensemble()
        POUR chaque élément dans this
            résultat.ajouter(élément)
        POUR chaque élément dans otherSet
            résultat.ajouter(élément)
        RETOURNER résultat
FIN
```

13. Arbre AVL

Définition

Arbre binaire de recherche auto-équilibré où la hauteur des sous-arbres diffère au plus de 1.

Cas d'utilisation

- Quand pire cas $O(\log n)$ garanti
- Bases de données en mémoire
- Dictionnaires
- Plus complexe qu'un BST simple

Complexité: Toutes opérations $O(\log n)$

```
MÉTHODE équilibrer(nœud)
    équilibre = hauteur(gauche) - hauteur(droit)
    SI équilibre > 1 // Gauche lourd
        SI hauteur(gauche.gauche) >= hauteur(gauche.droit)
            rotationDroite(nœud)
        SINON
            rotationGaucheDroite(nœud)
    SI équilibre < -1 // Droit lourd
        SI hauteur(droit.droit) >= hauteur(droit.gauche)
            rotationGauche(nœud)
        SINON
            rotationDroiteGauche(nœud)
```

14. Arbre Rouge-Noir

Définition

Arbre binaire de recherche auto-équilibré avec propriétés de coloration.

Cas d'utilisation

- Implémentation de TreeMap/TreeSet
- Systèmes de fichiers
- Bases de données
- Programmation concurrente

Propriétés

1. Chaque nœud est rouge ou noir
2. La racine est noire
3. Les feuilles (NULL) sont noires
4. Un nœud rouge a deux enfants noirs
5. Tous chemins d'un nœud à ses feuilles ont même nombre de nœuds noirs

Complexité: Toutes opérations $O(\log n)$



Tableau Comparatif des Complexités

Structure	Accès	Recherche	Insertion	Suppression
Tableau	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Liste Chaînée	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Pile	$O(1)^*$	$O(n)$	$O(1)$	$O(1)$
File	$O(1)^*$	$O(n)$	$O(1)$	$O(1)$
Table Hachage	$O(1)$	$O(1)$	$O(1)$	$O(1)$
BST	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Guide de Sélection

Pour accès rapide par index

- ☒ Tableau
- ☒ Tableau Dynamique

Pour insertions/suppressions fréquentes

- ☒ Liste Chaînée
- ☒ Liste Doublement Chaînée

Pour recherche rapide par clé

- ☒ Table de Hachage
- ☒ Arbre de Recherche

Pour données ordonnées

- ☒ Arbre Binaire de Recherche
- ☒ Tas

Pour relations complexes

- ☒ Graphe

Pour gestion FIFO/LIFO

- ☒ Pile/File